`Rotator` Script Explained

This Unity component spins a GameObject around one axis at a set speed. Rotation can be started, stopped, or toggled from UI buttons, and by default it spins around the object's visual center instead of its pivot.

Settings (Inspector)

Field	Purpose
`axis`	Which local axis to spin around: X, Y, or Z
`rotationSpeed`	Speed in degrees per second (default 30)
`startRotating`	Whether it spins as soon as the scene loads
`rotateAroundCenter`	Spin around the center of the object's renderers rather than its pivot point

How It Works

1. Start
Copies `startRotating` into the private `isRotating` flag, which is the live on/off switch.

2. Update (every frame)

- If `isRotating` is false, it returns immediately.
- Otherwise it computes `angle = rotationSpeed × Time.deltaTime`. Multiplying by `deltaTime` makes the speed frame-rate independent.
- Then it picks one of two paths:

Mode	Method used	Behavior
`rotateAroundCenter = true`	`transform.RotateAround(center, worldAxis, angle)`	Spins around the visual center, even if the model's pivot is off to one side
`rotateAroundCenter = false`	`transform.Rotate(axis, angle, Space.Self)`	Spins around the object's own pivot

In center mode, the local axis is converted to world space with `TransformDirection`, because `RotateAround` expects a world-space axis.

Helper Methods

`GetWorldCenter()`

- Gathers all `Renderer` components on the object and its children.
- Merges their bounding boxes with `Bounds.Encapsulate` into one box.
- Returns that box's center.
- If there are no renderers, it falls back to `transform.position`.
- It runs every frame, so the center stays accurate if the object moves, animates, or scales.

`GetRotationAxis()`

Converts the `axis` enum into a vector: X → `Vector3.right`, Y → `Vector3.up`, Z → `Vector3.forward`.

Public Methods (for UI Buttons)

- `ToggleRotation()` flips the on/off state.
- `StartRotation()` turns rotation on.
- `StopRotation()` turns rotation off.

Hook any of these to a Button's `OnClick()` event in the Inspector.

VR Note

The closing comment says no changes are needed for VR. A future controller or XR interaction can call `ToggleRotation()` just like the desktop button does.

Things Worth Knowing

- **Performance:** `GetComponentsInChildren<Renderer>()` allocates a new array every frame in center mode. For one object that's fine, but for many rotators, cache the renderers in `Start()` (and refresh the cache only if children change).
- **Drift risk:** Using `RotateAround` with a bounds center that is recomputed each frame can cause slight positional wobble, because the bounds of a rotating object (axis-aligned) change as it turns. For non-symmetrical meshes, you may notice the object subtly shifting. Caching the center once, or using a local-space center offset, avoids this.
- **Axis space:** In both modes the chosen axis is the object's local axis, so if the object is already tilted, it spins around its tilted axis.

`CenterScreenUIInteractor` Script Explained

This Unity component lets a first-person player press a key to click world-space UI buttons by aiming the screen center (crosshair) at them. It fakes a mouse pointer fixed to the middle of the screen, highlights the button under it, and fires the button's `OnClick()` when the interaction key is pressed.

Settings (Inspector)

Field	Purpose
`interactionDistance`	Maximum distance (default 5) at which a button can be targeted
`interactionKey`	Key that activates the targeted button (default E)

The key's type depends on which input system the project uses. With the new Input System (`ENABLE_INPUT_SYSTEM`), it's a `Key`; otherwise it's a legacy `KeyCode`. This is handled with `#if` compiler directives.

Private State

- `pointerEventData`: a reusable fake pointer event, positioned at screen center.
- `currentButton`: the button currently under the crosshair, or null.

How It Works

1. Start

- Checks that an `EventSystem` exists in the scene. If not, it logs an error and stops.
- Otherwise it creates the `PointerEventData` used for all raycasts.

2. Update (every frame)

- Returns early if there's no `EventSystem`.
- Calls `FindButtonAtScreenCenter()` to see what's under the crosshair.
- If a button is targeted and the interaction key was pressed this frame, it calls `currentButton.onClick.Invoke()`.

Helper Methods

`FindButtonAtScreenCenter()`

- Sets the fake pointer position to `(Screen.width / 2, Screen.height / 2)`.
- Calls `EventSystem.current.RaycastAll` to collect everything UI-related under that point.
- Loops through the results and skips any with `distance > interactionDistance`.
- For each remaining hit, it searches upward with `GetComponentInParent<Button>()`. This matters because the ray usually hits a child (like the button's text or image), not the Button component itself.
- It takes the first button that is `interactable` and stops.
- Passes the result (possibly null) to `UpdateHighlightedButton()`.

`UpdateHighlightedButton(Button newButton)`

- If the target hasn't changed, it does nothing.
- If there was a previous button, it sends it a `pointerExit` event, which removes the hover look.
- Stores the new button.
- If there is one, it sends it a `pointerEnter` event, which triggers the hover/highlight look.

This reuses the Button's normal hover visuals (color tint, sprite swap, animation) without a real mouse.

`InteractPressed()`

- New Input System: returns false if no keyboard exists, otherwise `wasPressedThisFrame` for the chosen key.
- Legacy Input: returns `Input.GetKeyDown(interactionKey)`.

Typical Setup Requirements

- An `EventSystem` in the scene.
- A World Space Canvas with a **Graphic Raycaster** (the default raycaster for UI).
- For world-space canvases, the canvas needs its **Event Camera** set to the player camera so raycasts work correctly.
- Buttons whose `OnClick()` calls other scripts, such as `Rotator.ToggleRotation()` from the previous script.

VR Note

The closing comment says this is only the desktop pointer implementation. In VR, the XR Interaction Toolkit's controller ray would provide the pointer and click, so this script would no longer be needed.

Things Worth Knowing

- **Missing exit event on click-away:** `pointerExit` and `pointerEnter` are sent with the same shared `pointerEventData`, which is fine for visuals but means it doesn't track real press/release state. It only invokes `onClick` directly, so it bypasses `pointerDown`/`pointerUp` visuals (the "pressed" tint won't show).
- **Distance caveat:** For world-space canvases, `result.distance` is measured from the event camera to the hit point. This works as intended for distance limits, but screen-space overlay canvases report a distance of 0, so they'd always pass the check.
- **Allocation per frame:** `new List<RaycastResult>()` is created every frame. Making it a reusable field and calling `results.Clear()` avoids garbage collection churn.
- **Stale button if disabled:** If the highlighted button is destroyed or disabled while targeted, `currentButton` can become a dead reference until the next frame's check replaces it. This is usually harmless because the next `FindButtonAtScreenCenter()` call reassigns it.
- **Hits behind other UI:** The loop takes the first valid button in raycast order, so a non-button element in front doesn't block a button behind it unless you add a check for that.